CURSO DE BIG DATA E IA · SESIÓN 2

Inteligencia Artificial

Cuatro técnicas que aún no hemos visto, con casos reales.

1 · Reglas

2 · Aprendizaje automático

3 · Stable Diffusion

4 · Ética y ley

Contenidos basados en los módulos 5071 y 5072 (BOE núm. 111, 10 de mayo de 2021)

SESIÓN 2 · QUÉ VEMOS HOY

Cuatro técnicas que aún no hemos visto

Reglas de conocimiento

Una persona experta escribe el criterio y el sistema lo aplica.

Caso real: XCON configuraba ordenadores en DEC.

Aprendizaje automático

El modelo aprende el criterio a partir de ejemplos.

Caso real: Zillow valoraba y compraba casas.

Redes neuronales

Capas que aprenden solas qué mirar en imágenes, texto o proteínas.

Caso real: AlphaFold, Nobel de Química 2024.

Stable Diffusion

Crea imágenes nuevas quitando ruido paso a paso.

Caso real: imágenes falsas y análisis forense.

Ya vimos en la sesión 1: refuerzo con Trackmania, tokens y RAG, agentes y coches autónomos. Hoy abrimos la caja.

ANTES DE EMPEZAR · DEFINIR EL PROBLEMA

Requisitos de un sistema de resolución de problemas

1

Objetivo

Qué se quiere conseguir.

2

Entrada

Datos y estado de partida.

3

Acciones

Qué puede hacer el sistema.

4

Criterio

Cómo saber si funciona.

El criterio decide la técnica: si se puede escribir, reglas; si solo hay ejemplos, aprendizaje automático; si son imágenes, texto o audio, redes neuronales; si hay que crear contenido nuevo, IA generativa.

ANTES DE EMPEZAR · ELEGIR TÉCNICA

Cuatro técnicas: cuándo usar cada una

Reglas de conocimiento

Cuando el criterio se puede explicar y hay que justificar cada decisión.

Necesita: personas expertas.

Aprendizaje automático

Cuando hay muchos ejemplos con la respuesta conocida.

Necesita: datos etiquetados.

Redes neuronales

Cuando los datos son imágenes, audio o texto.

Necesita: muchos datos y GPU.

Stable Diffusion

Cuando hay que crear imágenes nuevas a partir de un texto.

Necesita: GPU y revisión humana.

Cómo elegir: la técnica más sencilla que cumpla el criterio de éxito. Las reglas incluyen la automatización de tareas, la lógica difusa y los sistemas expertos.

01

Reglas de conocimiento

Cuando una persona experta puede explicar el criterio: automatización, razonamiento impreciso, reglas y sistemas expertos.

Automatización de tareas

Ejemplo: registrar facturas recibidas por correo (automatización robótica de procesos, RPA).

Llega el correo → **Extrae importe y fecha** → **Registra en contabilidad**

Ventaja: rápida, sin errores de copia y trazable.

Límite: si cambia el formato de la factura, el proceso se rompe.

PARTE 1 · RESOLUCIÓN DE PROBLEMAS

Razonamiento impreciso: lógica difusa

En vez de «hace calor: sí o no», cada temperatura tiene un grado de pertenencia entre 0 y 1 a cada categoría.

Temperatura	Frío	Templado	Caliente
18 °C	0,7	0,3	0,0
22 °C	0,0	1,0	0,0
27 °C	0,0	0,4	0,6

Regla difusa: si está caliente, ventilador rápido; si está templado, ventilador lento.

A 27 °C: mezcla ambas reglas y da una velocidad intermedia.

PARTE 1 · RESOLUCIÓN DE PROBLEMAS

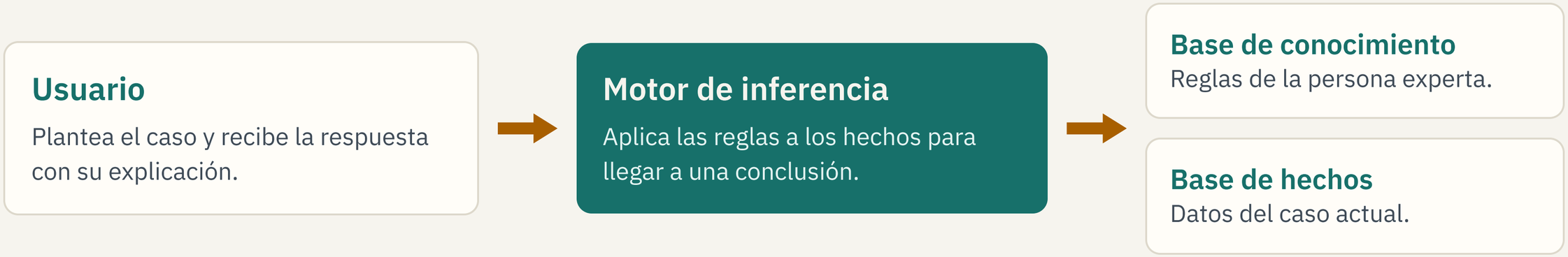
Sistemas basados en reglas

```
def derivar(asunto):
  if "factura" in asunto:
    return "Contabilidad"
  if "contraseña" in asunto:
    return "Soporte"
  return "Revisión manual"
```

Ventajas: explicables, auditables y sin datos de entrenamiento.

Inconvenientes: hay que mantenerlas a mano y no aprenden solas.

Estructura de un sistema experto



Quién la construye: el ingeniero del conocimiento entrevista a la persona experta y traduce su criterio a reglas.

Estrategias de control: encadenamiento

```
R1: si no enciende y cable conectado → fallo de fuente
R2: si fallo de fuente → llamar a técnico
```

Hacia delante

Parte de los hechos

Hechos: no enciende, cable conectado. Se dispara R1 y luego R2: llamar a técnico.

Hacia atrás

Parte de la hipótesis

Hipótesis: llamar a técnico. Pide R2, que exige fallo de fuente, y eso exige comprobar R1.

PARTE 1 · PRÁCTICA EN COLAB

Un mini sistema experto en Python

```
reglas = [
  ({"no_enciende", "cable_ok"}, "fallo_fuente"),
  ({"fallo_fuente"}, "llamar_tecnico"),
]
hechos = {"no_enciende", "cable_ok"}
cambio = True
while cambio:
  cambio = False
  for cond, concl in reglas:
    if cond ⩽ hechos and concl not in hechos:
      hechos.add(concl); cambio = True
print(hechos)
```

1. Ejecuta y comprueba qué hechos nuevos aparecen.

2. Añade una regla y un hecho nuevos.

3. Imprime cada regla que se dispara.

PARTE 1 · CASO REAL

XCON: el sistema experto que configuraba ordenadores

1980
entra en producción en DEC, escrito en OPS5

10.129
reglas en 1989, frente a 6.200 en 1987

95–98 %
de pedidos bien configurados en 1986

≈ 25 M$
de ahorro estimado al año

Qué hacía: revisaba cada pedido de ordenadores VAX y completaba piezas, cables y armarios antes de fabricarlo.

El límite: cada producto nuevo obligaba a cambiar reglas, y mantenerlas era cada vez más costoso.

PARTE 1 · EJERCICIO

Ejercicio: ¿qué modelo elegirías?

Caso A

Pasar a contabilidad facturas con siempre el mismo formato.

Caso B

Regular un ventilador según una temperatura que cambia de forma gradual.

Caso C

Justificar a un cliente por qué se le deniega una póliza según criterios escritos.

Opciones: automatización, razonamiento impreciso o sistema basado en reglas. Justifica con los cuatro requisitos del problema.

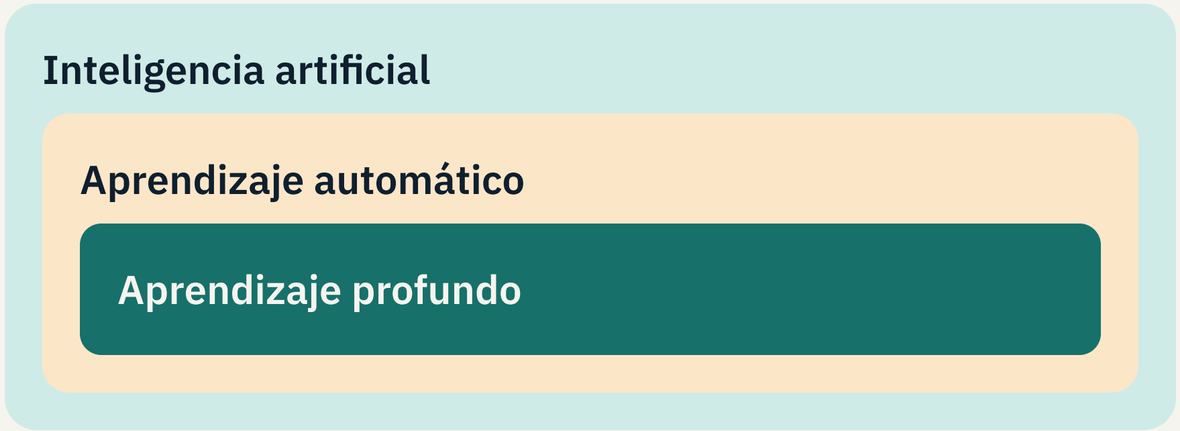
02

Aprendizaje automático y redes neuronales

Modelos que aprenden de los datos. Casos reales: Zillow y AlphaFold.

PARTE 2 · IA, ML Y TIPOS

IA, aprendizaje automático y aprendizaje profundo



IA: cualquier técnica que imita capacidades inteligentes.

ML: aprende patrones de los datos en vez de usar reglas escritas.

Profundo: ML con redes neuronales de muchas capas.

PARTE 2 · IA, ML Y TIPOS

Tres tipos de aprendizaje

Supervisado

Aprende de ejemplos con respuesta conocida (etiqueta).

Ejemplo: predecir si un correo es spam.

No supervisado

Busca estructura en datos sin etiqueta.

Ejemplo: agrupar clientes parecidos.

Por refuerzo

Un agente prueba acciones y recibe recompensas.

Ejemplo: un robot que aprende a caminar.

Procedimiento común: datos, identificar patrones, tomar decisiones.

PARTE 2 · TÉCNICAS POR TIPO

Qué técnicas usa cada tipo de aprendizaje

Tipo	Técnicas y algoritmos	Dónde lo vemos
Supervisado	Clasificación y regresión, árboles de decisión, razonamiento basado en casos, redes neuronales	Zillow, árboles, AlphaFold
No supervisado	Agrupación (k-means), reducción de dimensión (PCA), detección de anomalías	Más adelante en esta parte
Por refuerzo	Q-learning y su versión con redes neuronales (DQN)	Trackmania, en la sesión 1

Aprendizaje inductivo: pasar de ejemplos concretos a una regla general. Es la base de todo el supervisado.

Las redes neuronales no son un tipo más: sirven en los tres, según cómo se entrenen.

PARTE 2 · TRES TIPOS DE PROYECTO

Mismos pasos, distinto contenido

Paso	Supervisado	No supervisado	Por refuerzo
Datos	Históricos con la respuesta: ventas cerradas, correos marcados como spam	Históricos sin respuesta: compras de clientes, lecturas de sensores	Los genera el propio agente al actuar en un entorno o simulador
Limpieza y ETL	Quitar duplicados y nulos, unir fuentes, crear variables	Lo mismo, y escalar las variables para medir distancias	Definir estados, acciones y recompensa
Entrenar	Ajustar el modelo para acertar la etiqueta	Buscar grupos, resúmenes o rarezas	Probar acciones y mejorar la estrategia (tabla Q)
Evaluar	Con datos de test: acierto, precisión, error	Con criterio de negocio: ¿los grupos tienen sentido?	Recompensa acumulada en episodios nuevos
Producción	Predice cada caso nuevo	Asigna cada caso a su grupo o avisa de una rareza	Decide en el entorno real, con límites de seguridad

Lo que cambia es de dónde sale la respuesta correcta: de los datos, de nadie o de una recompensa.

PARTE 2 · ANALOGÍA CON BIG DATA

Un proyecto de IA es un proyecto de datos

PROYECTO DE BIG DATA · BLOQUES 1 A 4

Ingesta
Ficheros, APIs, eventos

→

Bronce
Datos crudos

→

Plata
Limpios, con ETL en Spark

→

Oro
Agregados por negocio

→

Cuadro de mando
Una persona decide

PROYECTO DE IA · HOY

Ingesta
Las mismas fuentes

→

Bronce
Datos crudos

→

Plata
Limpios, con ETL en Spark

→

Oro
Tabla de entrenamiento

→

Modelo
Predice o decide solo

Lo nuevo va al final: entrenar, evaluar y registrar el modelo (en Databricks, con MLflow), y vigilarlo en producción.

El bucle: los datos nuevos vuelven a entrar por bronce y, si el modelo empeora, se reentrena. Lo veremos con el caso de Zillow.

PARTE 2 · SUPERVISADO EN DETALLE

El supervisado, en ocho fases

1. **Elegir algoritmo**
2. **Seleccionar datos**
3. **Construir el modelo**
4. **Validarlo**
5. **Ajustar parámetros**
6. **Probar con datos nuevos**
7. **Implementar**
8. **Optimizar**

Lo hacemos paso a paso con datos reales: predecir el valor de la vivienda en 20.640 zonas de California (censo de EE. UU. de 1990).

PARTE 2 · PASO 1 · ELEGIR ALGORITMO

Clasificación y regresión

Paso 1 de 8

Elegir algoritmo

Clasificación

La salida es una categoría.

Ejemplos: spam o no spam; fraude o pago normal.

Regresión

La salida es un número.

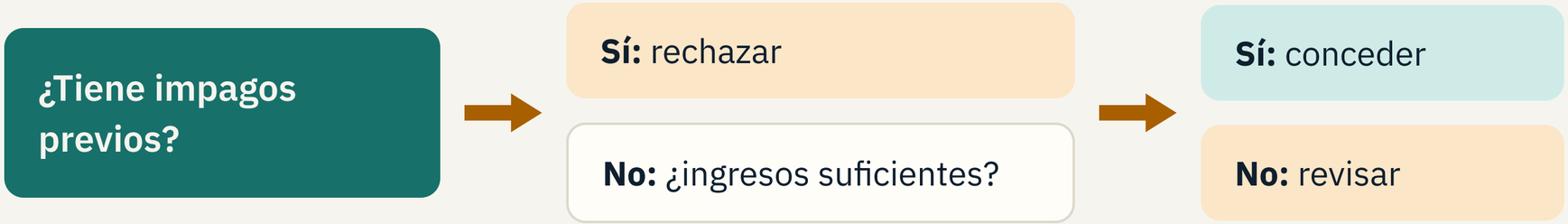
Ejemplos: precio de una vivienda; demanda de la semana próxima.

Regresión lineal, la más sencilla:

```
valor = w1 · renta + w2 · antigüedad + … + b
```

Nuestro caso: el valor de una vivienda es un número, así que es regresión. Probaremos dos candidatos: regresión lineal y árbol de decisión.

Candidato 2: árbol de decisión



Ventaja: cada rama se lee como una regla si-entonces.

Riesgo: un árbol muy profundo memoriza los datos.

Datos etiquetados: viviendas de California

Renta media	Antigüedad	Habitaciones	Latitud	Longitud	Etiqueta: valor
83.300 $	41 años	6,98	37,88	−122,23	**453.000 $**
33.800 $	29 años	4,84	37,69	−121,76	**184.000 $**
36.800 $	33 años	4,82	32,74	−117,04	**141.000 $**
18.900 $	39 años	5,23	36,04	−119,40	**112.000 $**

20.640 filas, una por zona del censo, con 8 variables de entrada (también dormitorios, población y ocupación).

Etiqueta: valor mediano de la vivienda en la zona. Es lo que el modelo debe aprender a predecir.

Construir el modelo: qué hace fit()


```
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
modelo.fit(X_ent, y_ent)

modelo.coef_ # un peso por variable
modelo.intercept_ # el término b
```

1 · Empieza

Con todas las filas de entrenamiento y sus etiquetas.

2 · Mide el error

Compara la predicción con el valor real en cada fila.

3 · Ajusta los pesos

Busca los que hacen mínimo el error al cuadrado.

Resultado real: valor ≈ 0,44 · renta – 0,42 · latitud – 0,43 · longitud + … Cada 10.000 $ más de renta en la zona suman unos 44.000 $ al valor, con el resto igual.

PARTE 2 · PASO 4 · VALIDAR

Entrenamiento, validación y test

Paso 4 de 8

Validar

Entrenamiento · 70 % · 14.448 zonas | Valid. · 3.096 | Test · 3.096

Entrenamiento

El modelo aprende de estos datos.

Validación

Sirve para comparar y ajustar parámetros.

Test

Evaluación final, una sola vez.

Validar: ¿cuánto se equivoca?

91.000 $
Línea base: predecir siempre la media

53.000 $
Regresión lineal

49.500 $
Árbol con 6 niveles

La métrica

Error absoluto medio (MAE): cuánto se aleja, de media, la predicción del valor real.

La línea base

Si un modelo no mejora a «decir siempre la media», no sirve para nada.

Con datos de validación

Medido sobre las 3.096 zonas que el modelo no vio al entrenar.

Matriz de confusión

100 correos	Predice spam	Predice no spam
Real: spam	40 · acierto (VP)	10 · se escapa (FN)
Real: no spam	5 · falsa alarma (FP)	45 · acierto (VN)

Falso positivo (FP): un correo bueno marcado como spam.

Falso negativo (FN): un spam que llega a la bandeja.

Métricas a partir de la matriz

Exactitud

`(VP+VN) / total`

85 %

(40+45) / 100

Precisión

`VP / (VP+FP)`

88,9 %

40 / 45

Exhaustividad

`VP / (VP+FN)`

80 %

40 / 50

Cuidado: si el 99 % de los casos es «normal», un modelo que siempre dice «normal» tiene 99 % de exactitud y no sirve.

Capacidad de generalización

Situación	Error en entrenamiento	Error en test	Qué hacer
Subajuste	Alto	Alto	Modelo más potente o más variables
Buen ajuste	Bajo	Bajo, parecido	Es el objetivo
Sobreajuste	Muy bajo	Alto	Más datos o modelo más simple

Sobreajuste: el modelo memoriza los ejemplos en vez de aprender el patrón, y falla con datos nuevos.

Ajustar parámetros: ¿cuántos niveles?

Niveles del árbol (max_depth)	Error en entrenamiento	Error en validación	Lectura
2	65.200 $	65.200 $	Subajuste
6	47.600 $	49.500 $	Mejorando
10	31.800 $	**43.300 $**	**El mejor**
15	12.000 $	44.200 $	Empieza a memorizar
Sin límite	0 $	47.600 $	Sobreajuste

El parámetro se elige con validación, nunca con test. Un bosque de 200 árboles (random forest) baja el error de validación a unos 33.700 $.

Probar con datos nuevos: una sola vez

90.400 $
Línea base

53.500 $
Regresión lineal

32.500 $
Random forest

Cómo

Se reentrena con entrenamiento y validación juntos y se mide una sola vez en las 3.096 zonas de test.

Qué pesa más

Renta media (53 %), ocupación (14 %) y ubicación (18 % entre latitud y longitud).

Regla

Si ahora retocáramos el modelo mirando el test, ese número dejaría de ser honesto.

Implementar: llevar el modelo al trabajo


```
import joblib
joblib.dump(modelo, "valor_v1.joblib")

# en producción
modelo = joblib.load("valor_v1.joblib")
modelo.predict(nuevas_zonas)
```

Por lotes

Cada noche puntúa las zonas nuevas de la capa plata y escribe en oro (un job de Databricks).

En tiempo real

Una API recibe una vivienda y devuelve su valor en milisegundos.

Versionar: guardar con el modelo qué datos, código y parámetros lo produjeron. En Databricks se hace con el registro de modelos de MLflow.

Optimizar: vigilar y mejorar

Medir en real

Cuando se vende la casa, comparar el precio real con la predicción.

Vigilar la deriva

Si cambian los datos de entrada (tipos de interés, rentas), el modelo envejece.

Reentrenar

Con los datos nuevos, por calendario o cuando el error supera un umbral.

Mejorar

Nuevas variables (cercanía al mar, colegios) o un algoritmo mejor.

El ciclo vuelve a empezar: los datos nuevos entran por bronce y el modelo se revisa. Lo siguiente es un caso real de lo que pasa si no se hace a tiempo.

PARTE 2 · PASO 8 · OPTIMIZAR · CASO REAL

Zillow: cuando el modelo de precios falla

Paso 8 de 8

Optimizar

2018

Zillow empieza a comprar casas según lo que valoran sus modelos

381 M$

de pérdidas de ese negocio en el tercer trimestre de 2021

25 %

de la plantilla despedida al cerrarlo en noviembre de 2021

Regresión en producción

El modelo estimaba el precio de cada casa, como el ejemplo de la tabla de viviendas.

El mercado cambió

Los precios dejaron de seguir el patrón de los datos de entrenamiento.

El error se multiplica

Un error pequeño por casa, repetido en miles de compras, es una pérdida enorme.

PARTE 2 · NO SUPERVISADO Y REFUERZO

Aprendizaje no supervisado

No hay etiqueta: el algoritmo busca estructura por sí mismo en los datos.

Agrupación

Clientes con hábitos parecidos.

Reducción de dimensión

Resumir muchas variables en pocas.

Anomalías

Detectar lo que no encaja.

Dificultad: sin etiquetas no hay «respuesta correcta»; hay que interpretar y validar los resultados con criterio de negocio.

Agrupación con k-means

1

Elegir k y colocar k centros.

2

Asignar cada punto al centro más cercano.

3

Mover cada centro a la media de su grupo.

4

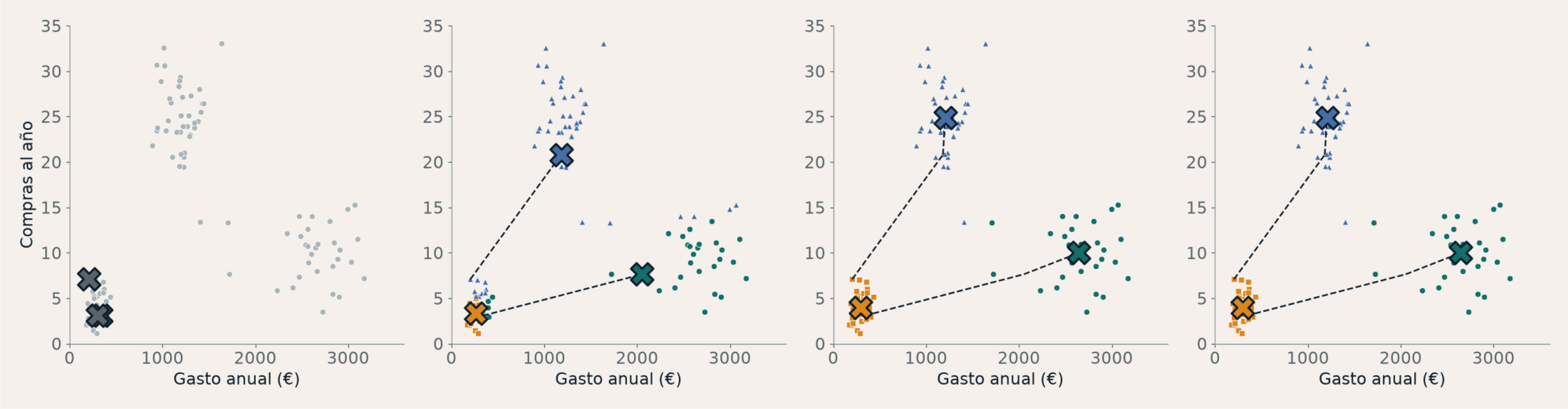
Repetir hasta que los centros no cambien.

Ejemplo: clientes según gasto y frecuencia de compra, en k = 3 grupos.

Límite: hay que decidir k antes y el resultado depende del punto de partida.

PARTE 2 · NO SUPERVISADO Y REFUERZO

k-means paso a paso, con clientes

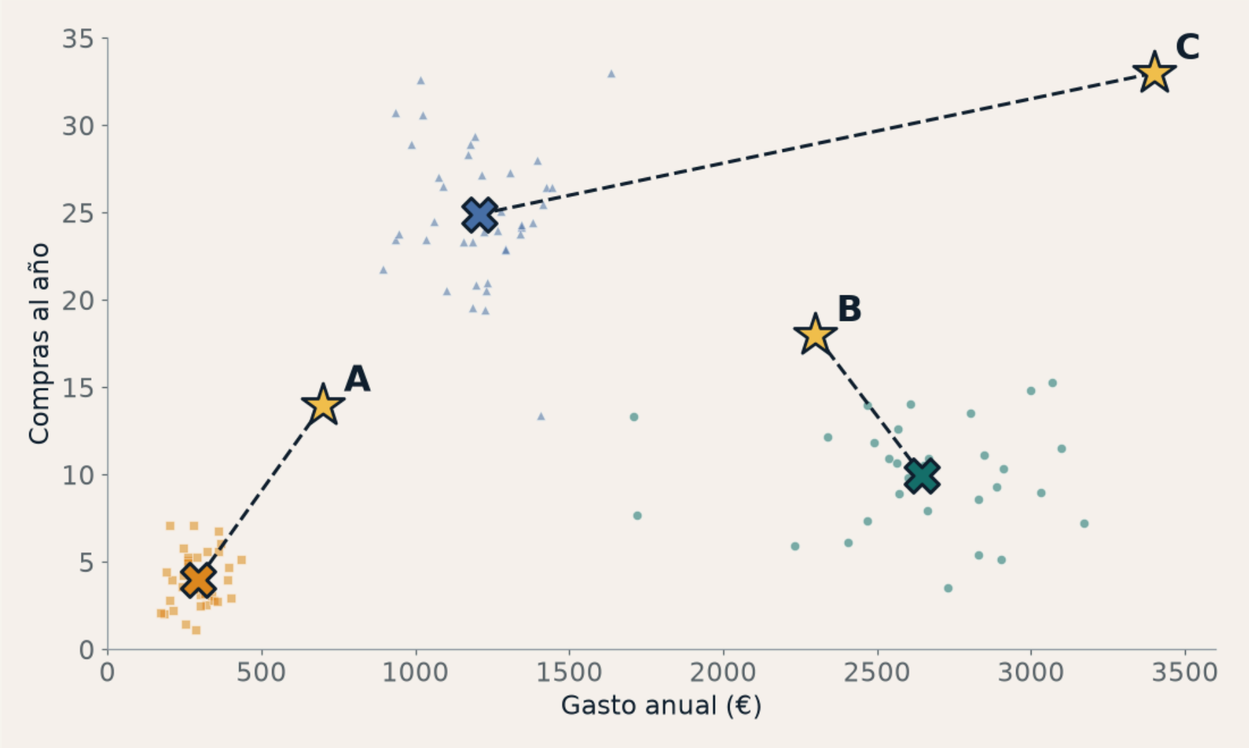


Inicio: 3 centros al azar | **Iteración 1: asignar y mover** | **Iteración 2** | **Iteración 3: ya no se mueven**

Tres grupos que nadie había etiquetado: ocasionales (≈ 290 € y 4 compras al año), frecuentes (≈ 1.200 € y 25) y grandes compras (≈ 2.650 € y 10).

PARTE 2 · NO SUPERVISADO Y REFUERZO

¿Y si llega un cliente nuevo?



B · 2.300 € y 18 compras

Cae dentro de «grandes compras»: se asigna sin dudas.

A · 700 € y 14 compras

Su centro más cercano es «ocasionales», pero está fuera del grupo.

C · 3.400 € y 33 compras

El más cercano es «frecuentes», pero está lejos de todos: ¿anomalía o un grupo nuevo?

Reducción de dimensión

Muchas variables
Por ejemplo, 50 columnas por cliente.

→

PCA
Combina variables en componentes que recogen la mayor variación.

→

Pocas variables
Dos o tres, aptas para dibujar.

Para qué: visualizar, reducir ruido y acelerar otros modelos.

Coste: los componentes son más difíciles de interpretar que las variables originales.

PARTE 2 · NO SUPERVISADO Y REFUERZO

Detección de anomalías

Se aprende cómo es lo normal y se marca lo que se aleja de ello.

Usos

Fraude con tarjeta, fallo de un sensor, intrusión en una red.

Cómo

Por distancia al resto, por densidad o por desviación estadística.

Reto

Hay muy pocos casos anómalos y casi nunca están etiquetados.

Conexión con Big Data: con flujos de eventos continuos se detectan anomalías casi en tiempo real.

Aprendizaje por refuerzo: el bucle

1 Observa el estado
¿Dónde estoy? La casilla del circuito.

2 Elige una acción
Explora al azar o usa lo que ya sabe.

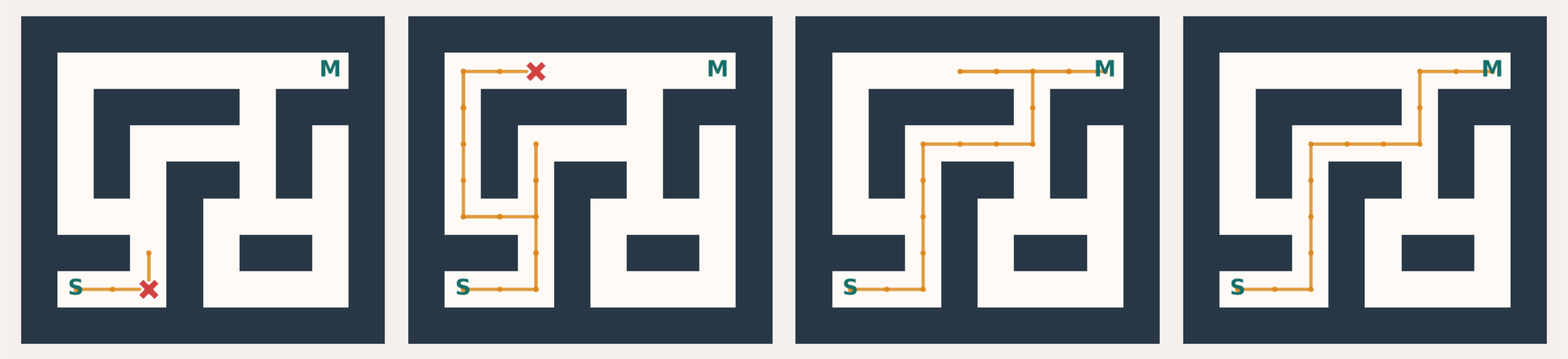
3 El entorno responde
−1 por paso, −10 si choca, +100 en la meta.

4 Actualiza la tabla Q
Corrige cuánto vale esa acción en ese estado.

Un episodio: dar vueltas al bucle hasta llegar a la meta o chocar. Después, otro episodio desde la salida.

PARTE 2 · NO SUPERVISADO Y REFUERZO

Episodio a episodio, el agente encuentra el camino



Episodio 1 · explora al azar y choca · −14

Episodio 175 · llega lejos y choca · −34

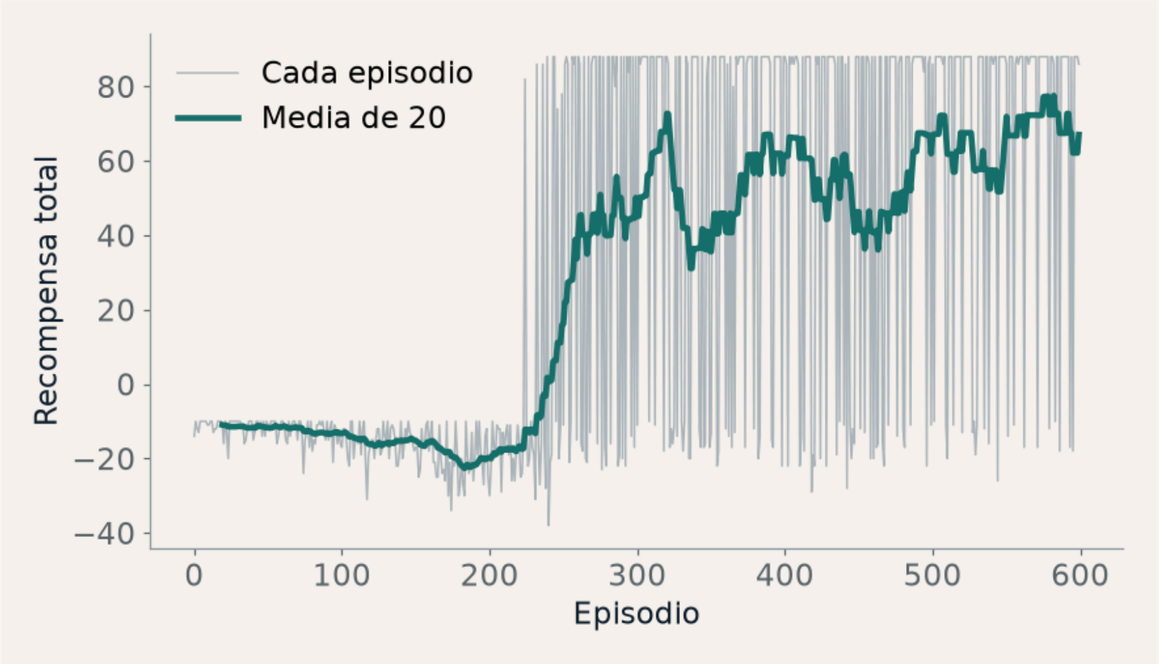
Episodio 225 · primera meta, 19 pasos · +82

Al final · el camino más corto, 13 pasos · +88

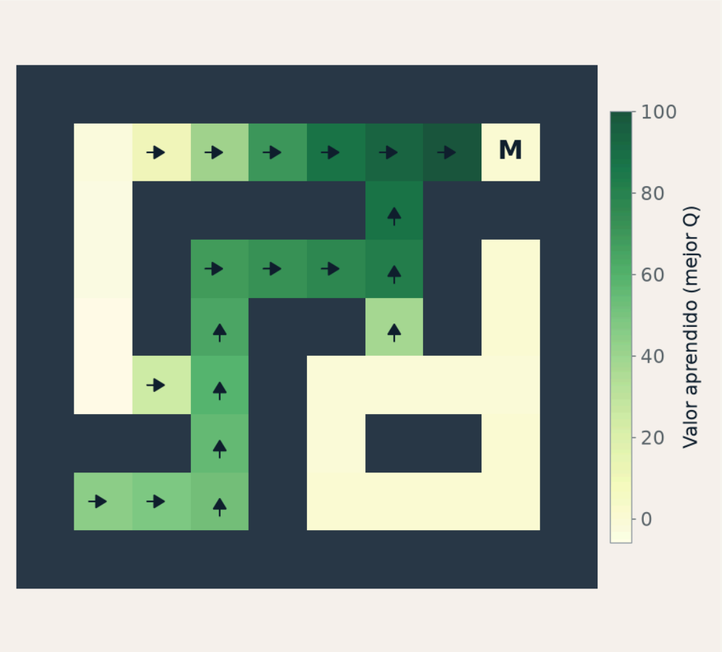
Explorar o aprovechar: al principio elige al azar el 99 % de las veces; episodio a episodio explora menos (hasta el 5 %) y usa más lo que ha aprendido.

IA · Sesión 2 · Parte 2 · Laberinto inventado; Q-learning con α = 0,5 y γ = 0,95, como el script 06; la cruz roja marca dónde chocó

Qué ha aprendido: la curva y la tabla



La recompensa sube cuando encuentra la meta. Los picos hacia abajo son exploraciones que acaban en choque.



La tabla Q, dibujada: la flecha es la mejor acción y el verde, cuánto vale estar ahí.

PARTE 2 · NO SUPERVISADO Y REFUERZO

El agente aprende de cada experiencia nueva

Una experiencia: estado, acción, recompensa, nuevo estado

$$Q(s,a) \leftarrow Q + \alpha \cdot (r + \gamma \cdot \max Q(s') - Q)$$

```
Q actual = 2,0
recompensa r = -1 (choca)
mejor Q(s') = 5,0
objetivo = -1 + 0,95 · 5 = 3,75
nuevo Q = 2 + 0,5 · (3,75 - 2) ≈ 2,88
```

Al momento

Cada paso corrige una celda de la tabla Q. No hay que reentrenar desde cero.

Con memoria

Las redes (DQN) guardan experiencias y repasan lotes mezclados.

Con cuidado

Se aprende en simulador; en el mundo real, con límites de seguridad.

PARTE 2 · RESUMEN

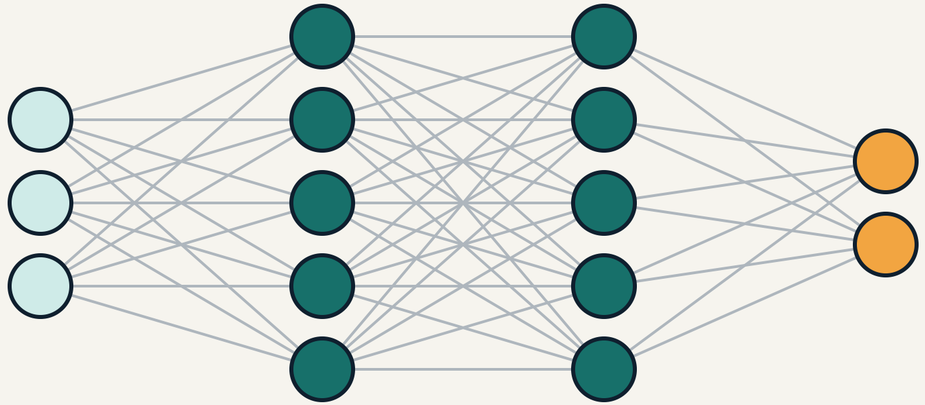
¿Qué pasa cuando llegan datos nuevos?

Tipo	Al llegar el dato	Cómo aprende de él	Cuándo reentrenar
Supervisado	Predice al momento	Solo cuando se conoce la etiqueta (el precio real, si fue fraude)	Por calendario o si el error sube (Zillow)
No supervisado	Se asigna al grupo más cercano	Puede mover los centros poco a poco (partial_fit)	Si aparecen muchos casos lejos de todos los grupos
Por refuerzo	Decide una acción	Corrige su tabla o red con la recompensa, al momento	Aprende siempre; se vigila que no aprenda algo peligroso

En los tres casos los datos nuevos entran por la misma tubería de Big Data: bronce, plata y oro.

PARTE 2 · REDES NEURONALES · INTRODUCCIÓN

¿Qué es una red neuronal?



Neuronas

Cada círculo suma sus entradas multiplicadas por unos pesos y aplica una función.

Capas

Las ocultas transforman los datos paso a paso; la salida da la respuesta.

Aprender

Es ajustar los pesos con ejemplos hasta que la salida acierte.

Hitos: perceptrón (1958) · retropropagación (1986) · AlexNet (2012) · Transformer (2017)

Redes en los tres tipos de aprendizaje

Tipo	Qué aprende la red	Ejemplo real
Supervisado	A acertar una etiqueta: qué hay en una imagen, qué forma tiene una proteína	Visión por computador, AlphaFold
Sin etiquetas	A predecir partes de los propios datos, como la siguiente palabra de un texto	El preentrenamiento de los modelos de lenguaje
Por refuerzo	A elegir acciones que dan más recompensa	La DQN en Atari, AlphaGo y el ajuste de ChatGPT con valoraciones humanas

¿Es lo más usado? En imágenes, texto, audio e IA generativa, sí: casi todo son redes.

En datos de tabla (ventas, clientes), los árboles como random forest o XGBoost siguen ganando a menudo.

PARTE 2 · REDES NEURONALES

Cómo aprende una red neuronal

Una neurona:

```
salida = activación(w1·x1 + w2·x2 + b)
```

1

Predecir

Los datos atraviesan las capas.

2

Medir el error

Se compara con la etiqueta real.

3

Propagar hacia atrás

Se calcula cuánto contribuyó cada peso.

4

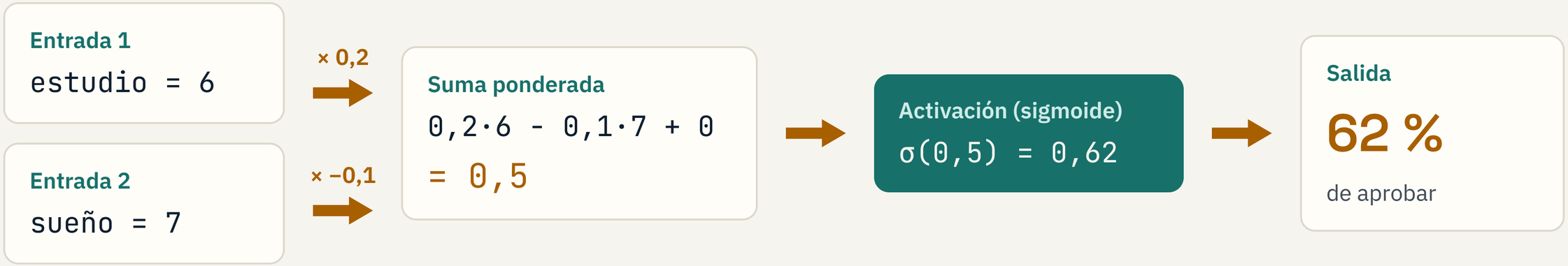
Ajustar pesos

Y se repite muchas veces.

PARTE 2 · REDES NEURONALES · PASO 1 · PREDECIR

Una neurona, con números

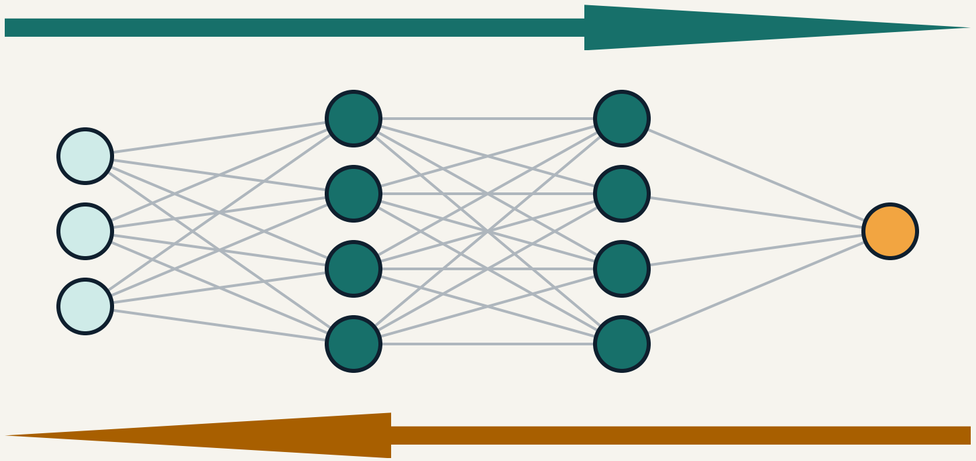
Ejemplo de juguete: ¿aprobará Ana? Estudió 6 horas y durmió 7. La red empieza con pesos al azar.



Eso es todo lo que hace una neurona: multiplica cada entrada por su peso, suma y aplasta el resultado entre 0 y 1. Los pesos son lo que la red aprende.

Paso, época y entrenamiento: qué es cada cosa

Hacia delante: el ejemplo atraviesa todas las capas



Hacia atrás: el error vuelve y corrige cada peso

Paso

Ida y vuelta con un ejemplo (o un lote): predecir, medir el error, corregir pesos.

Época

Una pasada por todos los ejemplos. Con 40 alumnos de uno en uno, 40 pasos.

Entrenamiento

Muchas épocas seguidas, hasta que el error deja de bajar.

Entrenar no añade capas ni neuronas. La forma de la red se decide antes, como los niveles del árbol; al entrenar solo cambian los valores de los pesos.

PARTE 2 · REDES NEURONALES · PASOS 2 A 4 · APRENDER

Un paso de aprendizaje, con Ana

2 · Medir el error

`predicción - real`

`0,62 - 1 = -0,38`

Ana aprobó: la red se quedó corta.

3 · Repartir la culpa

`cambio = 0,05 · error · entrada`

Cada peso se corrige en proporción a lo que influyó.

4 · Ajustar y repetir

`nueva predicción: 89 %`

Más cerca del aprobado real.

Peso	Antes	Cambio	Después
Estudio	0,20	+0,05 · 0,38 · 6 = +0,11	**0,31**
Sueño	−0,10	+0,05 · 0,38 · 7 = +0,13	**0,03**
Término b	0	+0,05 · 0,38 = +0,02	**0,02**

PARTE 2 · REDES NEURONALES · CAPAS OCULTAS

¿Qué son las capas ocultas?

LA DAMOS NOSOTROS

Entrada

Los datos tal cual llegan.

Ana: 6 h de estudio y 7 h de sueño.

NADIE LA VE NI LA ETIQUETA

Capas ocultas

Combinaciones de la entrada que la red se inventa porque le ayudan a acertar.

Ana: algo parecido a «estudió bastante» y «descansó».

SE COMPARA CON LA RESPUESTA

Salida

La predicción final, que se contrasta con la etiqueta.

Ana: 62 % de aprobar; en realidad aprobó.

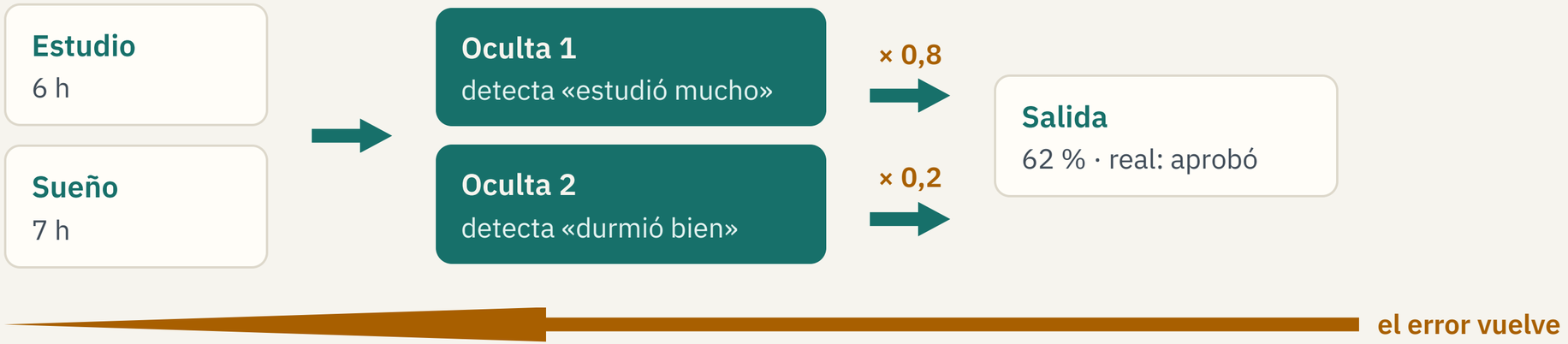
Con imágenes, cada capa oculta construye sobre la anterior:

Píxeles → **Bordes** → **Texturas y formas** → **Partes: ojos, orejas** → **«Gato»**

Se llaman ocultas porque no les damos ni les pedimos nada: solo vemos la entrada y la salida. Cuántas hay y de cuántas neuronas lo decidimos nosotros antes de entrenar.

PARTE 2 · REDES NEURONALES · HACIA ATRÁS

Con capas ocultas: el error viaja hacia atrás

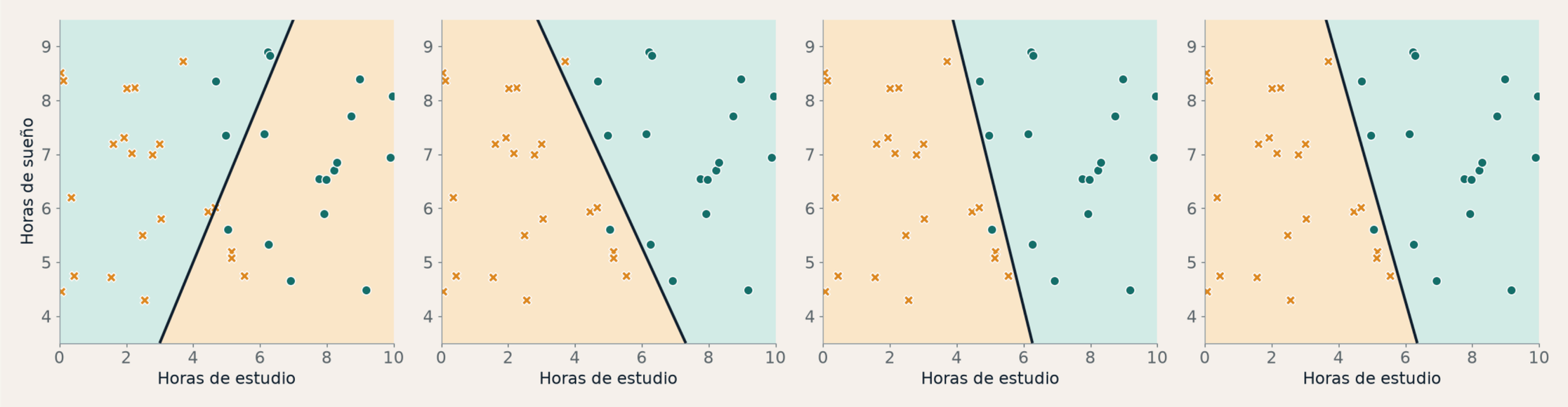


1 La salida mide su error (−0,38) y corrige sus dos pesos.

2 Reparte la culpa: la oculta 1 recibe más, porque su peso hacia la salida es mayor (0,8 frente a 0,2).

3 Cada neurona oculta corrige sus pesos de entrada con su parte de culpa.

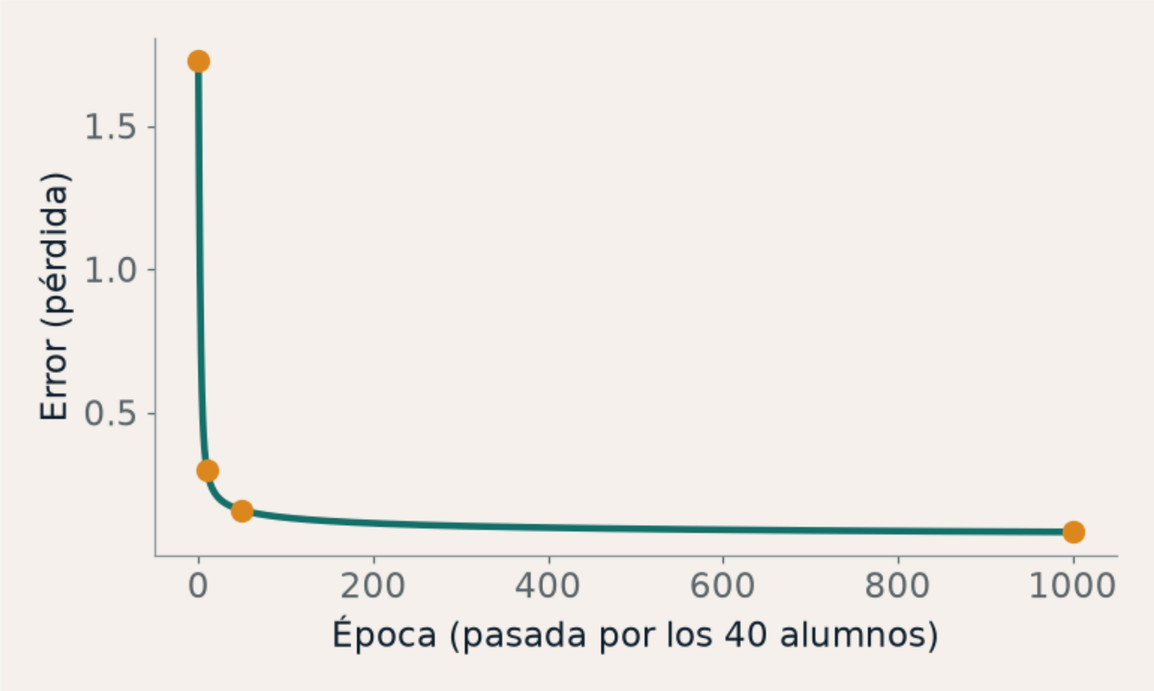
La frontera se mueve época a época



Época 0 · acierta el 18 % | **Época 10 · 95 %** | **Época 50 · 98 %** | **Época 1000 · 98 %**

Círculos verdes: aprueban. Cruces naranjas: suspenden. **La línea es donde la red duda (50 %)**; el color del fondo es lo que predice a cada lado.

El error baja época a época



Época

Una pasada completa por todos los datos. Aquí, 1000.

Tasa de aprendizaje

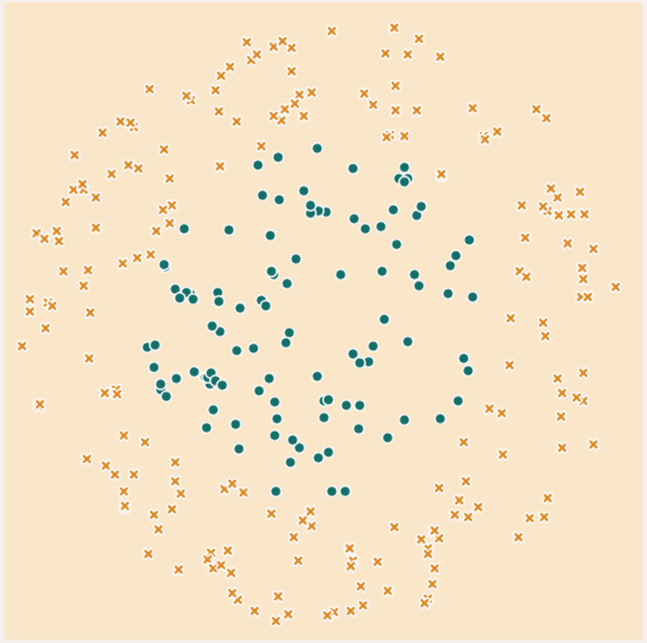
Cuánto se corrige en cada paso. Grande: rápido pero puede pasarse. Pequeña: segura pero lenta.

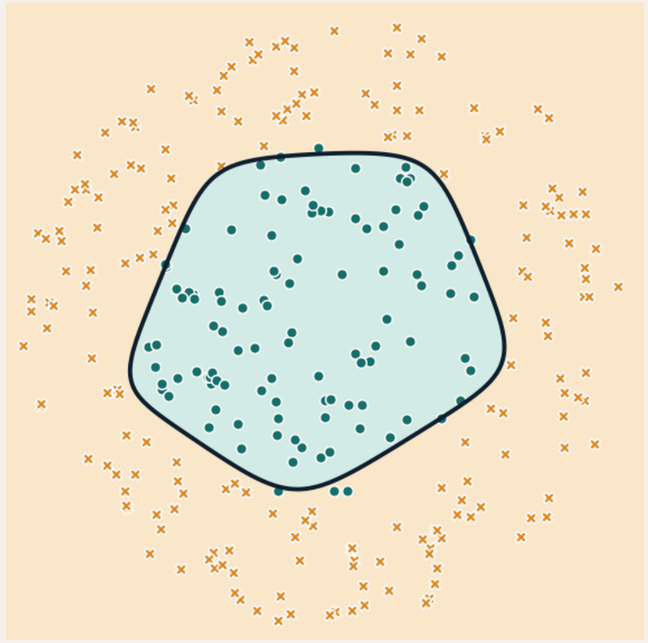
Cuándo parar

Cuando el error de validación deja de bajar: si se sigue, llega el sobreajuste.

PARTE 2 · REDES NEURONALES · CAPAS OCULTAS

Por qué hacen falta varias neuronas





Una neurona

Solo sabe dibujar una recta. Acierta el 65 %.

8 neuronas ocultas

Cada una dibuja una recta; la capa de salida las combina en una curva cerrada. Acierta el 98 %.

Más capas, formas más complejas: bordes, luego texturas, luego objetos. Es la idea de las redes convolucionales que vienen ahora.

IA · Sesión 2 · Parte 2 · 300 puntos inventados; scikit-learn, MLPClassifier con 8 neuronas ocultas

PARTE 2 · REDES NEURONALES · PRÁCTICA

Pruébalo en el navegador: TensorFlow Playground

1. Abre **playground.tensorflow.org** y elige el conjunto del círculo.
2. Quita todas las capas ocultas y pulsa ▶: la red solo dibuja una recta y no aprende.
3. Añade una capa con 4 neuronas: la frontera se cierra alrededor del círculo.
4. Prueba la espiral y sube la tasa de aprendizaje a 3: ¿qué le pasa a la curva de error?

Sin instalar nada. Es la misma idea de las slides anteriores, con los pesos dibujados en tiempo real.

Redes neuronales convolucionales (CNN)



Filtros: pequeñas ventanas que recorren la imagen y detectan bordes, texturas y formas.

Reducción: resume cada zona para trabajar con menos datos.

Usos habituales: reconocimiento de imágenes, control de calidad visual e imagen médica.

PARTE 2 · CASO REAL

AlphaFold: una red neuronal premiada con el Nobel

Secuencia
La cadena de aminoácidos de una proteína, como texto.

→

Red neuronal
AlphaFold2 (DeepMind, 2020) aprende de estructuras ya conocidas.

→

Estructura 3D
La forma de la proteína, que decide cómo funciona.

2024
Nobel de Química para Hassabis y Jumper, compartido con David Baker

≈ 200 M
de proteínas conocidas con la estructura predicha

> 2 M
de personas de 190 países lo han usado

PARTE 2 · LENGUAJE NATURAL

Procesamiento del lenguaje natural

El PLN permite que un sistema lea, interprete y genere lenguaje humano.

Clasificar

Spam, temas, sentimiento de una opinión.

Extraer

Nombres, fechas e importes de un texto.

Transformar

Traducir, resumir y reescribir.

Conversar

Asistentes y chatbots de atención.

Idea clave: el sistema no «entiende» como una persona: convierte el texto en números y busca patrones.

PARTE 2 · LENGUAJE NATURAL · EVOLUCIÓN

Del PLN con reglas a los LLM

1966

Reglas

Patrones escritos a mano.

Ej.: ELIZA, el primer chatbot.

→

2002

Estadística

Cuenta palabras y aprende de ejemplos.

Ej.: Filtros de spam bayesianos.

→

2013

Vectores

Cada palabra es un vector; las parecidas quedan cerca.

Ej.: word2vec, la base del RAG.

→

2017

Transformer y LLM

Redes enormes que leen el contexto entero.

Ej.: BERT (2018), ChatGPT (2022).

Son las técnicas de hoy, en orden: reglas (parte 1), aprendizaje automático y redes neuronales (parte 2). El PLN las ha usado todas.

PARTE 2 · LENGUAJE NATURAL · APRENDIZAJE AUTOMÁTICO CON TEXTO

Bolsa de palabras: un filtro de spam en una tabla

Mensaje	gratis	gana	premio	mañana	informe	Etiqueta
«Oferta gratis, gana dinero ya»	1	1	0	0	0	**Spam**
«Reunión mañana a las diez»	0	0	0	1	0	Normal
«Gana un premio gratis»	1	1	1	0	0	**Spam**
«Te envío el informe de mañana»	0	0	0	1	1	Normal

1 · Contar

Cada mensaje se convierte en números: cuántas veces sale cada palabra.

2 · Aprender

«gratis», «gana» y «premio» solo salen en spam; «mañana», en mensajes normales.

3 · Clasificar

«Premio gratis para ti»: dos palabras de spam y ninguna normal → spam.

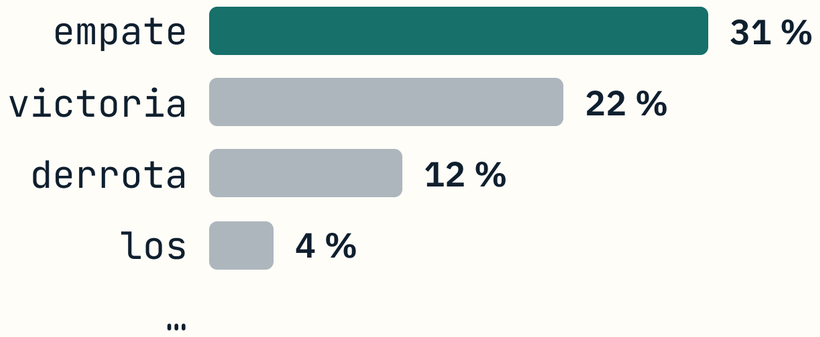
PARTE 2 · LENGUAJE NATURAL · CÓMO ESCRIBE UN LLM

Un LLM predice la siguiente palabra

Texto hasta ahora

«El partido terminó en …»

Probabilidad de la siguiente palabra



1 · Elige una palabra

Casi siempre de las más probables. La temperatura decide cuánto se arriesga.

2 · La añade y repite

«El partido terminó en empate» → siguiente palabra → … hasta acabar.

3 · Por eso inventa

Elige lo probable, no comprueba hechos: de ahí las alucinaciones.

PARTE 2 · LENGUAJE NATURAL

Potencial y limitaciones del PLN

Potencial	Limitación
Procesa miles de textos en poco tiempo	Ambigüedad, ironía y contexto
Traduce y resume con buena calidad media	Puede afirmar con seguridad algo falso
Se adapta a muchos dominios	Peor rendimiento en idiomas con pocos datos
Reduce trabajo manual repetitivo	Reproduce sesgos de los textos de partida

Papel del lingüista: define qué significa «correcto», anota ejemplos, evalúa la calidad y trabaja junto a informáticos.

PARTE 2 · ROBÓTICA

Sistemas robotizados

Percepción
Sensores y cámaras.

→

Planificación
Qué movimiento hacer.

→

Control
Ordena a los motores.

→

Actuación
El brazo se mueve.

Modelado cinemático: relaciona los ángulos de las articulaciones con la posición de la herramienta.

Programación: enseñando el movimiento al robot o escribiendo el código fuera de línea.

PARTE 2 · REDES NEURONALES

Aprendizaje automático frente a profundo

Aspecto	ML clásico	Aprendizaje profundo
Datos	Suficiente con volúmenes moderados	Suele necesitar muchos datos
Variables	Las diseña la persona	La red aprende sus propias representaciones
Tipo de dato	Tablas y registros	Imagen, audio y texto
Explicabilidad	Mayor, por ejemplo en árboles	Menor
Cómputo	Bajo o medio	Alto, con GPU

PARTE 2 · HERRAMIENTAS

Herramientas y plataformas del curso

Google Colab

Notebooks en la nube. Con scikit-learn para ML clásico y Keras o PyTorch para redes.

Databricks

Spark con datos grandes y MLlib para entrenar modelos a escala.

Edición gratuita de Databricks: solo cómputo serverless y sin GPU. Sirve para ML clásico con tablas; las redes neuronales, mejor en Colab.

PARTE 2 · PRÁCTICA EN COLAB

Un árbol de decisión con scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
X, y = load_iris(return_X_y=True)
Xe, Xt, ye, yt = train_test_split(X, y, test_size=0.3, random_state=0)
modelo = DecisionTreeClassifier(max_depth=3).fit(Xe, ye)
pred = modelo.predict(Xt)
print(accuracy_score(yt, pred))
print(confusion_matrix(yt, pred))
```

1. Ejecuta y lee la matriz de confusión.

2. Cambia max_depth y compara error en entrenamiento y en test.

3. Prueba KMeans con k = 3 sin usar las etiquetas.

03

IA generativa: Stable Diffusion

Cómo una red neuronal crea imágenes quitando ruido, y cómo se analiza después si una imagen es real.

PARTE 3 · QUÉ ES

Stable Diffusion: de un texto a una imagen

Prompt

«foto de un faro en la costa de Asturias al atardecer»

Imagen nueva

512 × 512 píxeles en Stable Diffusion 1.5, generada en segundos con una GPU.

Abierto

Agosto de 2022: código y pesos publicados. Funciona en una GPU doméstica con unos 5 GB de memoria.

Quién

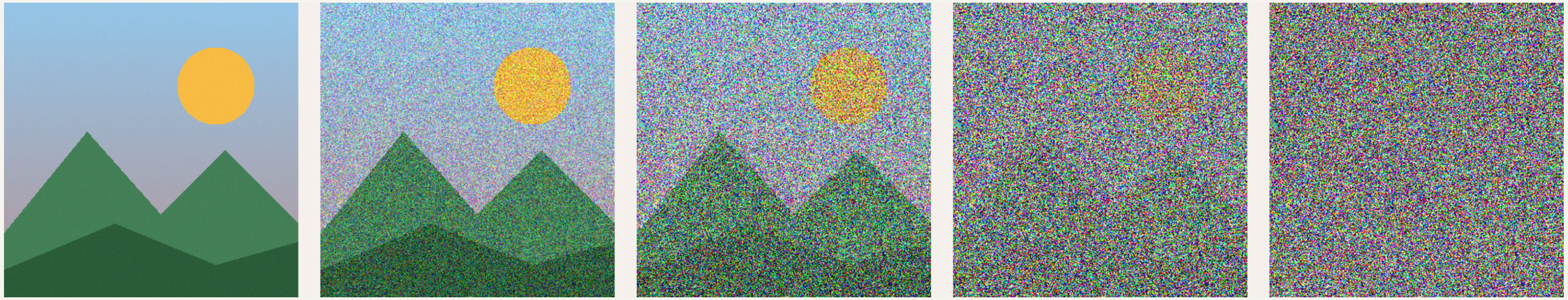
Stability AI, Runway y la LMU de Múnich, sobre la investigación de CompVis (Rombach y otros, 2022).

Con qué datos

Pares de imagen y texto de LAION-5B, recogidos de internet.

PARTE 3 · LA IDEA

Aprender a quitar ruido



Paso 0 · imagen | Paso 100 | Paso 250 | Paso 500 | Paso 1000 · ruido

Entrenar, de izquierda a derecha: se añade ruido a millones de imágenes reales y la red aprende a adivinar qué ruido se añadió.

Generar, de derecha a izquierda: se parte de ruido puro y la red lo quita poco a poco, guiada por el texto.

PARTE 3 · POR DENTRO

Tres redes que trabajan juntas

Texto
El prompt que escribe la persona.

→

Codificador de texto
CLIP convierte el texto en vectores.

→

U-Net
Quita ruido en el espacio latente, de 20 a 50 pasos.

→

Decodificador (VAE)
Pasa del espacio latente a los píxeles.

Semilla: el ruido inicial sale de un número. Misma semilla y mismos parámetros, misma imagen.

Espacio latente: la U-Net trabaja con 64 × 64 × 4 números en vez de 512 × 512 × 3 píxeles, unas 48 veces menos. Por eso es rápido.

PARTE 3 · STABLE DIFFUSION Y LOS LLM

Primos, no hermanos

Leen con el mismo motor

El codificador de texto de Stable Diffusion es un Transformer, la arquitectura de los LLM.

Cada vez más juntos

Stable Diffusion 3 (2024) añade un modelo de lenguaje, T5, y cambia la U-Net por un Transformer.

Trabajan en equipo

DALL·E 3 usa ChatGPT para convertir una idea en un prompt detallado antes de dibujar.

Un LLM escribe

Palabra a palabra, de izquierda a derecha, eligiendo la más probable.

Stable Diffusion dibuja

Toda la imagen a la vez, quitando ruido paso a paso, guiado por el texto.

En común: redes neuronales entrenadas con enormes cantidades de datos de internet. Por eso comparten riesgos: sesgos, derechos de autor y contenido falso que parece real.

PARTE 3 · PARÁMETROS

Lo que se puede ajustar al generar

Parámetro	Qué controla	Valor habitual
Prompt	Qué se quiere ver	Texto libre
Prompt negativo	Qué se quiere evitar	«borroso, texto»
Pasos	Cuántas veces se quita ruido	20 a 50
Escala de guía	Cuánto obedece al texto	Entre 7 y 8
Semilla	El ruido de partida	Un número
Resolución	Tamaño de la imagen	512 × 512 (versión 1.5)

Más pasos da más detalle pero tarda más. **Más guía** obedece más al texto, pero la imagen puede quedar forzada.

PARTE 3 · CASO REAL

Una imagen falsa que movió la bolsa

Qué pasó

22 de mayo de 2023: circula en Twitter una «foto» de una explosión junto al Pentágono. La bolsa cae brevemente.

Cómo se desmintió

Policía del Pentágono y bomberos: no hubo explosión. La fachada era incorrecta y no había más fotos ni testigos.

Qué exige la ley

El Reglamento de IA obliga a marcar el contenido generado y a avisar de las ultrafalsificaciones.

Lección: ver ya no es una prueba. Hay que comprobar el origen, buscar otras fuentes y analizar la propia imagen.

PARTE 3 · ANÁLISIS FORENSE

Buscar lo que no encaja

Metadatos

Una cámara deja marca y modelo. Muchas imágenes generadas no los tienen, o guardan el prompt.

Compresión (ELA)

Al recomprimir la foto, una zona pegada o retocada responde distinto.

Ruido

El sensor deja un ruido parecido en toda la foto. Lo pegado o generado rompe el patrón.

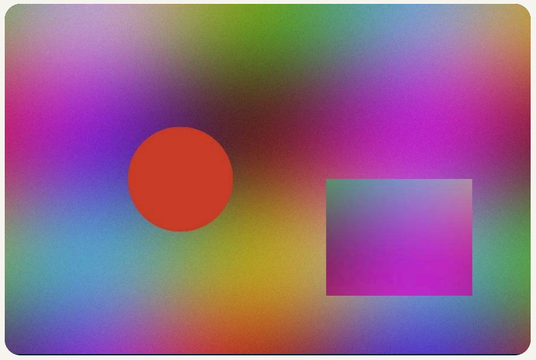
Frecuencias

Los generadores pueden dejar patrones repetidos en el espectro de la imagen.

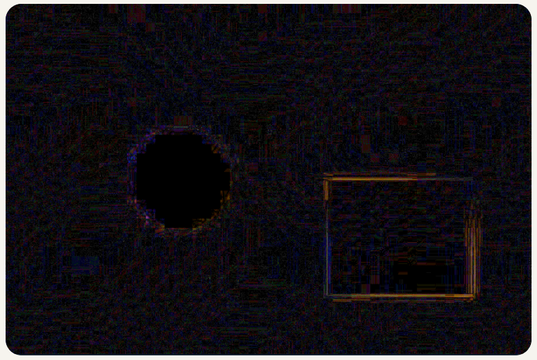
Sherloq reúne estas pruebas en una herramienta de código abierto. Ninguna es concluyente por sí sola: se combinan y se contrastan con el contexto.

PARTE 3 · EJEMPLO

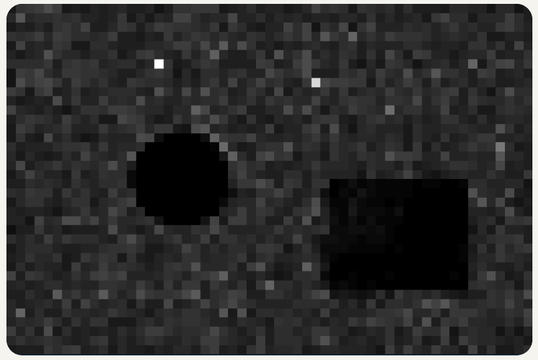
Una foto con una zona pegada



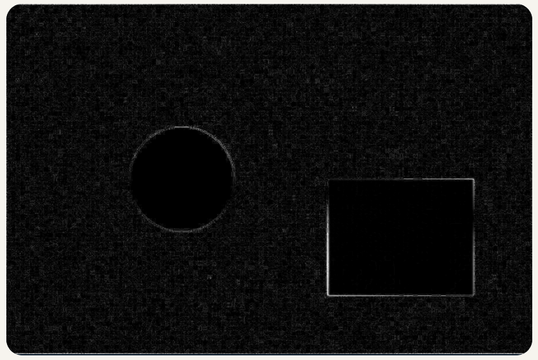
Original



ELA



Ruido por bloques



Alta frecuencia

El rectángulo de abajo se copió de arriba, se suavizó y se recomprimió. Aparece en las tres pruebas: borde marcado en ELA, menos ruido en el mapa de bloques y sin detalle fino.

PARTE 3 · PRÁCTICA EN LOCAL

Analizar una carpeta de fotos con Sherloq

```
# en el repositorio del curso
cd herramientas/forense
./analizar_fotos.sh ~/Desktop/fotos

# la aplicación completa
./analizar_fotos.sh --gui
```

1. La primera vez descarga Sherloq e instala Python 3.11 aparte.
2. Pasa nueve análisis a cada foto y lee sus metadatos.
3. Abre un informe en el navegador con todas las salidas.

¿Y en Colab o Databricks? Generar imágenes con Stable Diffusion sí se puede en Colab con GPU; Databricks gratuito no tiene GPU. Sherloq es una aplicación de escritorio, así que el análisis se hace en local.

04

Ética y marco legal

Riesgos, sesgos y normas que se aplican a cualquier sistema de IA.

Sesión 2 · Parte 4 · Módulos 5071 y 5072

PARTE 4 · ÉTICA Y MARCO LEGAL

Riesgos legales y éticos

Riesgo	Medida
Privacidad de los datos	Minimizar datos y anonimizar desde el diseño
Errores del sistema	Pruebas, supervisión humana y plan de respuesta
Ataques	Seguridad integrada en el diseño del sistema
Decisiones opacas	Explicar cómo se llega a cada resultado
Responsabilidad	Definir quién responde de cada decisión

Privacy by design: la privacidad se piensa desde el inicio.

Security by design: la seguridad también.

PARTE 4 · ÉTICA Y MARCO LEGAL

Sesgos de género en IA y Big Data

De dónde vienen

Datos históricos desbalanceados, etiquetas con prejuicios y equipos poco diversos.

Cómo se detectan

Midiendo el rendimiento del modelo por grupos, no solo la media global.

Cómo se corrigen

Auditar datos, reequilibrar, revisar variables y mantener revisión humana.

Caso Amazon (Reuters, 2018): un prototipo de selección entrenado con diez años de currículos, casi todos de hombres, penalizaba los que incluían la palabra «women's». Amazon lo abandonó.

Marco legal aplicable

Norma	Qué regula
RGPD (UE 2016/679)	Tratamiento de datos personales en la Unión Europea
LOPDGDD (LO 3/2018)	Protección de datos y derechos digitales en España
Reglamento de IA (UE 2024/1689)	Obligaciones según el nivel de riesgo del sistema

Reglamento de IA, idea general: cuanto mayor es el riesgo de un sistema, más obligaciones tiene; algunos usos están prohibidos.

Calendario: las prácticas prohibidas, desde febrero de 2025. Los sistemas de alto riesgo como selección de personal o crédito, desde el 2 de diciembre de 2027, tras el retraso del Reglamento (UE) 2026/1744.

CIERRE DEL BLOQUE

Cinco ideas para llevarse

1. Elige el modelo más sencillo que resuelva el problema.
2. Los datos de calidad pesan más que el algoritmo.
3. Evalúa siempre con datos que el modelo no ha visto.
4. Busca sesgos y explica las decisiones.
5. Cumple la privacidad y el marco legal desde el diseño.

Pregunta para el grupo: ¿qué proceso de vuestro entorno aplicaría estas cinco ideas primero?